

Task 4: Network Programming and IPC

Protocol Documentation and Testing Documentation — ST5004CEM Operating Systems and Security

Part A — Protocol Documentation

Transport: TCP (POSIX sockets), default port 5555. The protocol is a simple, human-readable, line-based text protocol: one command per line, one reply per line.

Command	Format	Server Response
Login	LOGIN <username> <password>	OK LOGIN / FAIL LOGIN
Send message	MSG <text>	ECHO <text>
List clients	LIST	CLIENTS <n>
Quit	QUIT	BYE (then connection closes)

Authentication and validation rules

- No command other than LOGIN is accepted until the client has successfully logged in; any attempt returns FAIL not logged in.
- Empty lines and lines longer than 500 characters are rejected with FAIL invalid command, without being processed further.
- Unrecognised commands return FAIL unknown command rather than being silently ignored.

Concurrency model

The server accepts connections in a loop and spawns a new std::thread for every accepted client, so the accept() loop is never blocked waiting on an existing client's conversation. This is what allows multiple clients to be served at the same time rather than one after another.

Part B — Testing Documentation

#	Test	Expected	Actual result
1	LOGIN with correct credentials	OK LOGIN	Pass — OK LOGIN received
2	LOGIN with incorrect credentials	FAIL LOGIN	Pass — FAIL LOGIN received
3	MSG sent before logging in	FAIL not logged in	Pass — command rejected
4	MSG after logging in	ECHO <text> returned	Pass — exact text echoed back
5	QUIT	BYE, then connection closes	Pass — socket closed cleanly
6	Multiple concurrent clients	Server reports correct live client count to each	Pass — see below

Concurrent clients — detailed result

Three client instances were started roughly one second apart, each pausing briefly after connecting before continuing. Because all three were connected simultaneously during that pause, each client's LIST command reported a different live count depending on how many were still connected at that moment:

- Client 1's LIST returned CLIENTS 3
- Client 2's LIST returned CLIENTS 2
- Client 3's LIST returned CLIENTS 1

This is direct evidence that the server was handling three separate client connections at the same time, rather than queueing and serving them one at a time. Each client received a live, independent response reflecting the true number of concurrently connected sockets at that instant.

Conclusion

All protocol commands were tested individually and behaved as specified, authentication and input validation were confirmed to reject invalid use correctly, and multiple concurrent client connections were demonstrated and verified through the LIST command's live client count.